



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 05 - Part 2

Classes and Objects in Detail





THIS WEEK'S AGENDA

- 01. Objects as Function Arguments
- 02. Returning Objects from Functions
- 03. Classes and Objects In Memory
- 04. Static Class Data
- 05. Revisiting Destructors





OBJECTS AS FUNCTION ARGUMENTS





OBJECTS AS FUNCTION ARGUMENTS

- Objects of any class can also be passed to functions as arguments, like other variables of primitive data types.
- Let's look at an example:

```
8  void Distance::add_dist(const Distance &d1, const Distance &d2) {
9      inches = d1.inches + d2.inches;
10     feet = 0;
11     if (inches >= 12.0) {
12         inches -= 12.0;
13         feet++;
14     }
15     feet += d1.feet + d2.feet;
16 }
```





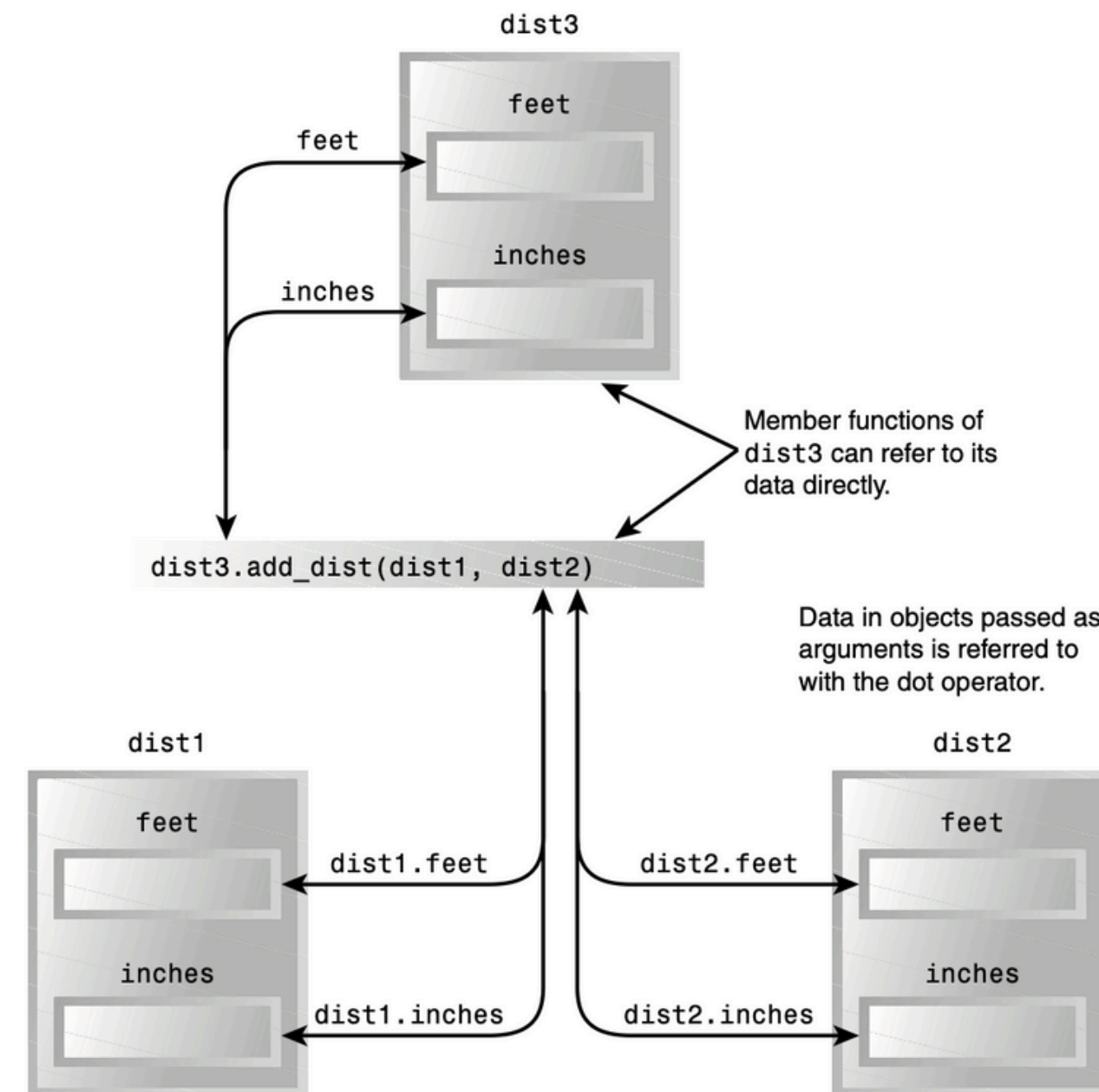
OBJECTS AS FUNCTION ARGUMENTS

- Now let's say we want to add `dist1` and `dist2` to obtain the result in `dist3`. We can then call the `add_dist` function in `main()` to do this:
 - `dist3.add_dist(dist1, dist2)`
- The two distances to be added, `dist1` and `dist2`, are supplied as arguments to the `add_dist()` member function of the `Distance` class.
- The syntax for arguments that are objects is the same as that for arguments that are simple data types such as integers (`int`), floats (`float`), booleans (`bool`), or characters (`char`).
- Since `add_dist()` is a member function of the `Distance` class, it can access the private data in any object of class `Distance` supplied to it as an argument.





OBJECTS AS FUNCTION ARGUMENTS





RETURNING OBJECTS FROM FUNCTIONS





RETURNING OBJECTS FROM FUNCTIONS

- Objects of any class can be returned from a function as well
- Let's look at an example:

```
18 Distance Distance::add_dist(const Distance &d) {
19     Distance temp;
20     temp.inches = inches + d.inches;
21     temp.feet = 0;
22     if (temp.inches >= 12.0) {
23         temp.inches -= 12.0;
24         temp.feet++;
25     }
26     temp.feet += feet + d.feet;
27     return temp;
28 }
```





OBJECTS AS FUNCTION ARGUMENTS

- Now let's say we want to add `dist1` and `dist2` to obtain the result in `dist3`. We can then call the `add_dist` function in `main()` to do this:
 - `dist3 = dist1.add_dist(dist2);`
- Now the `add_dist()` function is called from the `dist1` object and only the `dist2` object is passed as an argument instead of calling the `add_dist()` function from the resulting (`dist3`) object.
- A temporary `Distance` object, `temp`, returned from the `add_dist()` function which is then assigned to the `dist3` `Distance` object in `main`.





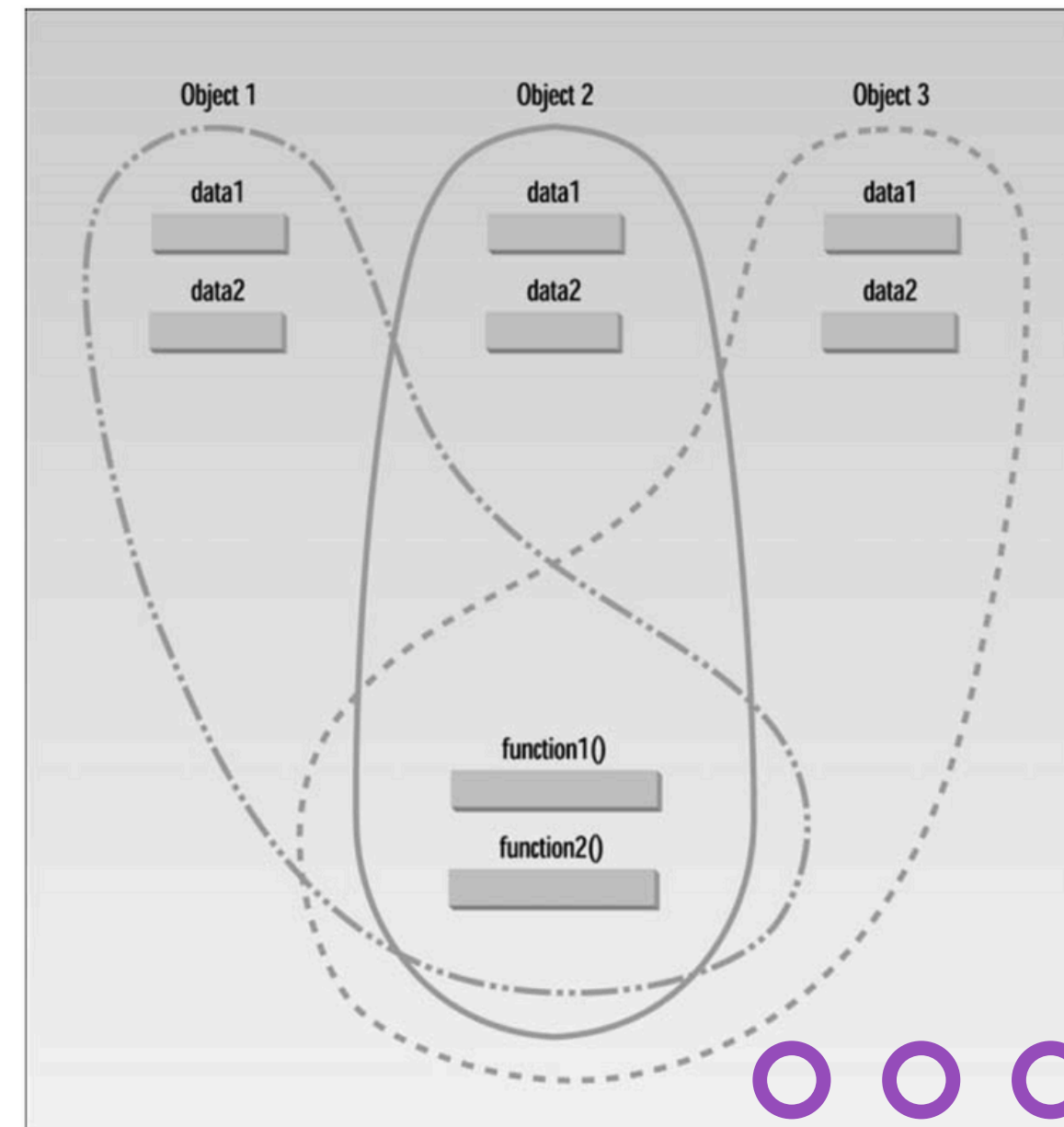
CLASSES AND OBJECTS IN MEMORY





OBJECTS AS FUNCTION ARGUMENTS

- In terms of internal memory layout,
 - All the objects of a particular class use the same member functions.
- These member functions are created and placed in memory only once at the time of being defined in the class definition.





STATIC CLASS DATA





STATIC CLASS DATA

- If we declare a static data item in a class, then only one such item is created for the entire class, rather than one for each object of the class, no matter how many objects there are.
- A static class item is useful when all objects of the same class must share a common piece of information, for example some sort of counter.
- A member variable defined as static has characteristics similar to a normal static variable.
- This static data item is visible only within the class, but has a lifetime of the entire program - it exists even if there are no objects of the class.





REVISITING DESTRUCTORS





DESTRUCTORS

- In order to deallocate memory allocated to an object, a special member function known as the destructor must be invoked.
- A destructor has to have the same name as the constructor (which is the same as the name of the class) but is preceded by a tilde.
- Let's look at an example:





DESTRUCTORS

```
3  class Distance {  
4      private:  
5          int feet;  
6          float inches;  
7      public:  
8          Distance();  
9          Distance(int ft, float in);  
10         ~Distance();  
11     };
```





SOME RULES FOR DESTRUCTORS

- Only one destructor can be defined for a class. It is not possible to define multiple destructors for a class.
- A destructor cannot be overloaded. It also cannot be declared as static or const.
- The destructor neither requires any argument nor returns any value (it does not have a return type).
- The destructor is automatically called when an object of a class goes out of scope.
- It releases the memory space occupied by the objects created by the constructor.
- Objects are destroyed in the reverse order of their creation.





SUMMARY





SUMMARY

- This session was a bit of an in-depth look at Object Oriented Programming.
- We looked at constructors in detail, including copy constructors and constructor overloading.
- We also looked at objects and how they can be used as function arguments.
- Objects can also be returned from functions.
- Static class data is a common piece of information that all objects of that class can access.
- Lastly, we looked at destructors and their functionality.





READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 06





ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

